

算法导论期末报告

姓名: 林晖鹏 学号: 2312966 专业: 计算机科学与技术

目录

1 问题背景	2
2 问题描述	2
2.1 具体细节	2
2.2 输入输出格式	2
3 算法设计与分析	3
3.1 state 状态和 Node 节点	3
3.1.1 状态定义	3
3.1.2 任务表示	4
3.2 dp 状态初始化	4
3.3 dp 状态转移	5
3.4 更新任务列表函数	6
3.5 结果计算	6
4 复杂度分析	6
4.1 时间复杂度	6
4.2 空间复杂度	6
5 程序结果	7
6 报告总结	7

摘要

本报告针对一名大二学生小林在期末前 30 天内优化学习计划的问题,设计并分析了一种基于动态规划的算法。小林需完成 8 个网课视频和 5 个大作业,任务间存在拓扑依赖关系。完成视频耗时 2 天得 5 分,大作业耗时 4 天得 30 分,提前完成任务需额外耗时(每未完成前置任务加 1 天)。未完成任务分别扣 5 分和 30 分,剩余时间可用于复习(每天 7 分)。初始得分为 300 分,目标是最大化总分。算法通过定义状态(天数、任务完成情况、得分、可执行/可跳过任务列表),构建拓扑图管理依赖关系,并通过动态规划实现状态转移,优化得分。时间复杂度为 $O(T \times N^3)$,空间复杂度为 $O(T \times N)$,其中 $T = 30$, $N = 13$ 。最终结果显示小林最多得 254 分,剩余 2 天复习。本文还讨论了参数设置对策略选择的影响及潜在优化方向。对现实选择有参考意义。

关键字: 动态规划, 拓扑排序, 任务调度, 优化算法, 学习计划

1 问题背景

小林是一名 NKU 计算机专业的大二学生，由于大二学业压力比较大，已经距离期末剩下一个月了，小林还有若干知识还没开始学、若干课程的大作业还没开始动。

他需要靠观看相应网课视频来补救知识点，而这些网课又可能存在一些依赖关系，一些网课是另一些网课的前置知识，不能够随便看；开始做大作业之前，又有一些前置知识需要观看网课来学习。如果没有把前置知识学习完，将会导致效率低下。

而如果到期末还有网课没看，那么面对对应知识点的题目，小林将一脸懵逼，导致期末考试分数偏低。如果还有大作业没有完成，那么这个课的平时分将会大残。

刚刚好算法导论课上翔哥教了动态规划，小林打算用 DP 来规划他接下来一个月的学习计划，尽可能要使得课程分数总和最大。

小林本人实力还不错，现在直接去考试，刚刚好每科能及格，但是期末的作业压力太大了，他还能及格吗？

2 问题描述

2.1 具体细节

如图2.1所示，小林还有八个网课视频还没有观看，还有五个大作业还没完成。小林目前总分能拿300分。但是还有三十天，小林需要尽可能要观看视频和完成作业来避免扣掉更多分。下面是题目设定：

- 大作业和网课视频之间存在如图2.1所示的拓扑前置关系。
- 期末之前，某个视频被跳过或者未观看，将会扣5分，某个大作业未完成，将会扣30分。
- 如果一个任务的所有前置任务都没有前置任务，那么这个任务可以被提前完成，也就是跳过所有未完成的前置任务，但是任务被跳过之后不会再回来完成。
- 看完一个网课视频的基础时间为2天，完成一个大作业的基础时间为4天；但是如果这些任务被提前完成，那么需要额外花费 **未完成的前置任务个数** 天。
- 小林也可以选择用剩余时间来复习，复习一天将能得到7分。

2.2 输入输出格式

由于上面的大部分定义在源代码的宏定义都写了

```
1 # define w_n 5 // 大作业个数
2 # define v_n 8 // 前置视频个数
3 # define work_time 4 // 完成一个大作业的基础时间
4 # define video_time 2 // 观看一个前置视频的基础时间
5 # define T 30 // 期末前的剩余时间
6 # define m 10 // 前置关系数
7 # define work_score 30 // 每个大作业的分数
8 # define video_score 5 // 每个视频的分数
```

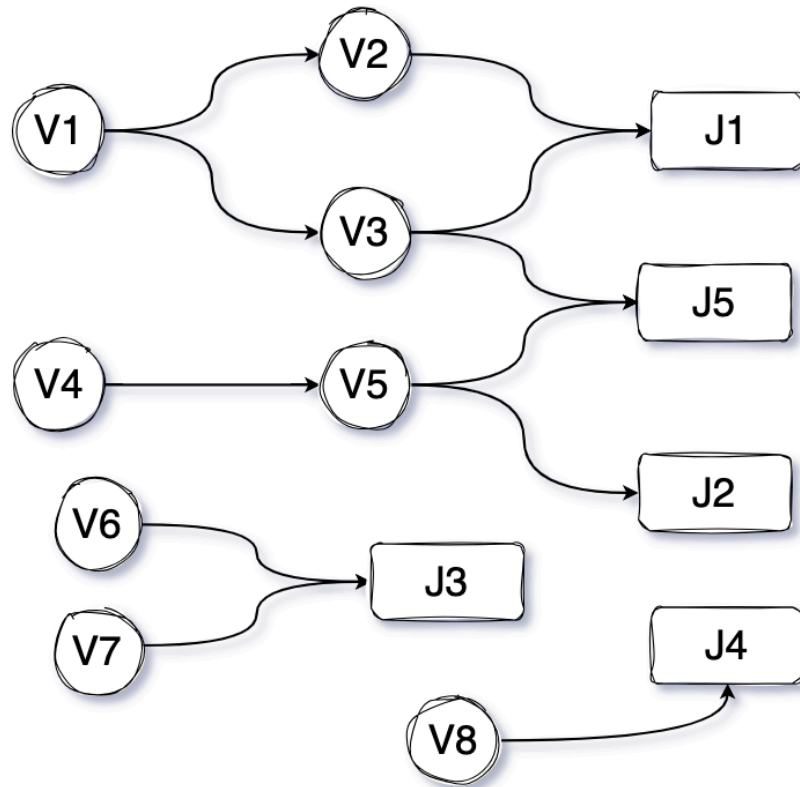


图 2.1: 拓扑示意图, 其中 V 代表网课视频、J 代表大作业

```

9 # define review_score 7 // 复习一天能获得的分数
10 # define basis_score 300 // 基础分数

```

所以这里只输入拓扑关系：

输入 m 行，每行两个整数，代表任务之间的前置关系。任务编号小于等于大作业个数的编号代表大作业，其余代表网课视频。输出一个整数，代表小林最终的得分。

3 算法设计与分析

本题目就是一维动态规划问题，但是附加的条件比较复杂。

我们用 dp 的思路，构建一个长度为 30 的 dp 状态数组，遍历到某一天的时候，遍历此状态下能完成或者能提前完成的任务，生成新的状态，同时更新本状态下的可完成可提前完成集合，然后与已用天数处的 dp 状态比较扣分，更新 dp 状态。最后得到最优复习策略。

3.1 state 状态和 Node 节点

这里设计了两个结构体，来存每个时刻的状态，以及利用节点来存储图拓扑信息。

3.1.1 状态定义

状态由结构体 state 表示，包含：

- `day`: 当前天数 (0 到 30)。
- `st`: 布尔数组, 记录任务 (编号 1 至 13, 其中 1-5 为大作业, 6-13 为视频) 的完成状态。
- `score`: 当前扣分, 初始为 $-(5 \times 30 + 8 \times 5) = -190$ (所有任务未完成的扣分)。
- `can_do`: 所有前置任务已完成的可执行任务列表。
- `can_skip`: 可提前完成的任务列表。

3.1.2 任务表示

任务由结构体 `Node` 表示:

- `id`: 任务编号 (1 至 13)。
- `is_video`: 布尔值, 判断是否为视频 (编号 6-13 为真)。
- `out, next`: 出度和后续任务列表。
- `in, pre`: 入度和前置任务列表。

3.2 dp 状态初始化

我们设计一个长度为 30 的 `dp` 状态数组, 初始化的时候, 每一天都当成完全没有完成任何作业的状态。同时统计一开始就能做的或者提前完成的任务。

1. 读取 $m = 10$ 条边, 构建依赖图, 填充 `nodes` 的 `next`、`pre`、`in` 和 `out`。
2. 入度为 0 的任务加入 `can_do`。
3. 前置任务无进一步前置任务的任务加入 `can_skip`。
4. 初始化 `dp[30]` 状态数组, 每一天 (0 到 29) 设置 `can_do`、`can_skip` 和初始得分。

这里是构建 `can_do`、`can_skip` 的核心代码:

```
1  vector<int> can_do; // 可以完成的任务列表
2  vector<int> can_skip; // 可以跳过的任务列表
3  // 初始化可以完成的任务列表
4  for(int i=1;i<=w_n+v_n;i++){
5      if(nodes[i].in == 0){
6          can_do.push_back(i); // 如果没有前置任务, 则可以完成
7      }
8      else{
9          bool can = 1; // 是否可以提前完成该任务
10         for(vector<int>::iterator it = nodes[i].pre.begin(); it != nodes[i].pre.end();
11             it++){
12             int pre_task = *it; // 前置任务编号
13             if(nodes[pre_task].in != 0) { //前面还有前置
```

```
13     can = 0;
14     break;
15 }
16 }
17 if(can) can_skip.push_back(i); // 如果可以提前完成, 则添加到可以完成的任务列表
18 }
19 }
20 //初始化状态
21 for(int i = 0; i < 30; i++) {
22     dp[i] = state(); // 每一天的状态初始化
23     dp[i].day = i; // 设置当前天数
24     dp[i].can_do = can_do; // 设置可以完成的任务列表
25     dp[i].can_skip = can_skip; // 设置可以跳过的任务列表
26 }
```

3.3 dp 状态转移

我们从 $i = 1, 2, \dots, 30$ 开始遍历 dp 数组, 在天数等于 i 的状态下, 我们遍历可完成可提前完成的集合, 然后生成新的状态, 根据扣分最少进行 dp 状态转移。同时需要更新新状态的可完成可提前完成集合。需要注意的是, 当提前完成任务, 需要统计被跳过的任务, 因为要额外花费时间。

对每一天 i (0 到 29):

1. **正常完成任务**: 对 $dp[i].can_do$ 中的每个任务:

- 复制 $dp[i]$ 创建新状态。
- 标记任务完成 ($st[task_id] = true$)。
- 大作业加 30 分, 视频加 5 分。
- 天数增加 4 (大作业) 或 2 (视频)。
- 从 can_do 移除该任务。
- 调用 $update_can_do$ 更新 can_do 和 can_skip 。
- 若 $new_state.day \leq 30$, 且得分更高, 更新 $dp[new_state.day]$ 。

2. **跳过任务**: 对 $dp[i].can_skip$ 中的每个任务:

- 复制 $dp[i]$ 创建新状态。
- 标记任务及其未完成的前置任务为已完成。
- 大作业加 30 分, 视频加 5 分。
- 天数增加 4 (大作业) 或 2 (视频) 加未完成前置任务数。
- 从 can_skip 移除该任务。
- 调用 $update_can_do$, 更新 can_do 和 can_skip 。
- 若 $new_state.day \leq 30$, 且得分更高, 更新 $dp[new_state.day]$ 。

3.4 更新任务列表函数

函数 `update_can_do` 在完成任务后更新 `can_do` 和 `can_skip`:

- 对每个后续任务 (`next_task`):
 - 若已完成, 跳过。
 - 检查是否 `next_task` 的所有前置任务已完成 (`can_to_do`)。
 - 检查是否 `next_task` 的所有前置任务在 `can_do` 中或已完成 (`can_to_skip`)。
 - 若 `can_to_do`, 加入 `can_do`; 若 `can_to_skip`, 加入 `can_skip`。

3.5 结果计算

遍历 `dp[0...29]`, 计算最大得分: $dp[i].score + (30 - i) \times 7$ (剩余天数用于复习)。记录最大得分和对应天数 `max_id`。输出:

- 最大总分: `basis_score + max_score`。
- 剩余复习天数: `30 - max_id`。
- 各任务完成情况 (大作业和视频)。

4 复杂度分析

4.1 时间复杂度

- **初始化**: 读取 $m = 10$ 条边, $O(m)$; 初始化 `can_do` 和 `can_skip`, $O(N+m)$, 其中 $N = w_n + v_n = 13$ 。
- **状态转移**: 对 30 天, 每天的 `can_do` 和 `can_skip` 最多包含 N 个任务。每次转移调用 `update_can_do`, 检查后续任务 (最多 N) 的前置任务 (最多 N), 复杂度为 $O(N^2)$ 。总转移次数为 $O(T \times N)$, 故总复杂度为 $O(T \times N^3)$, 其中 $T = 30$, $N = 13$ 。
- **结果计算**: $O(T)$ 。
- 总时间复杂度: $O(T \times N^3) \approx O(30 \times 13^3) \approx O(6.6 \times 10^5)$ 。

4.2 空间复杂度

- `nodes` 数组: $O(N + m)$ (存储前置和后续任务列表)。
- `dp` 数组: $O(T \times N)$ (每个状态存储 `st` 和任务列表)。
- 总空间复杂度: $O(T \times N) \approx O(30 \times 13) \approx O(390)$ 。

5 程序结果

最后剩下 2 天复习，最多能拿 254 分。作业完成情况如图5.2所示。

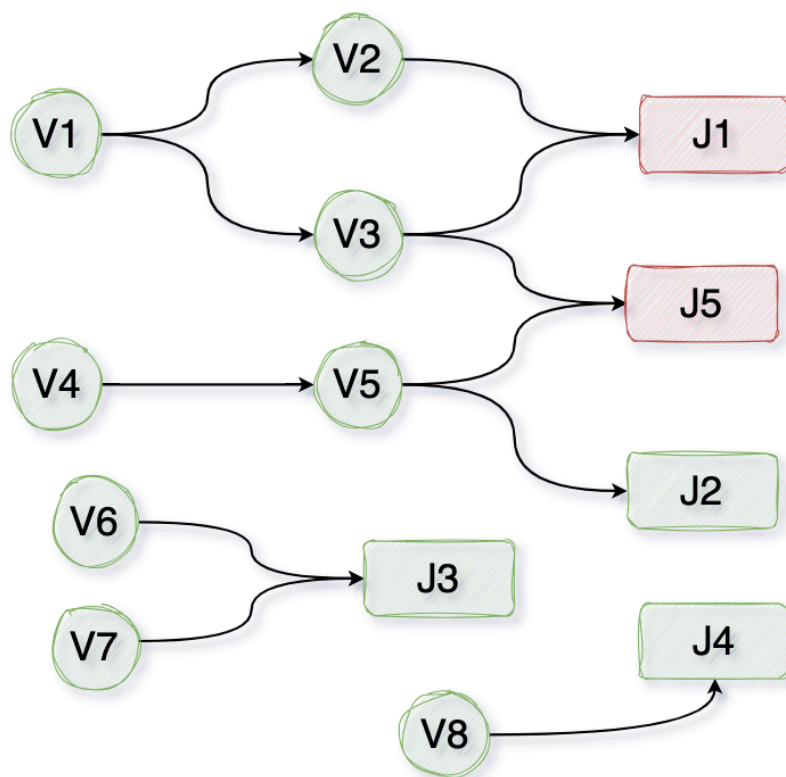


图 5.2: 完成情况，红色为未完成，绿色为完成

6 报告总结

看来这个期末是毁了，作业完不成，分也拿不到了呜呜呜呜。

本题目为本人出题，期末月压力太大了导致的灵感。但是其实题目中还有一些问题连出题人还没想清楚，要是读者想清楚了可以教教我：在状态转移的时候，不同策略花费的天数一样，得到的扣分也一样，此时应该选择转移还是不转移，一眼看去感觉就是 bug，但是想想，这样情况主要是提不提前完成任务导致的，但是提前完成任务的时间损失和分数损失在不同情况下都是一样的，那么到达这个天数的時候，如果分数一样，说明其实到达这里的策略是否在后续花费其实等价呢？

然后在设置题目参数的时候发现，分数的设置对策略的选择十分重要，在出题的时候本来以为会有提前完成的情况出现的，毕竟大作业分数比较高，但是看来 30 分还是不够，做大作业不如留时间复习和多看点网课。如果把大作业的分数占比提高，跳过的花销减少，估计会出现跳过情况。

然后我的实现还是有点粗糙的，首先是这里状态查询的时候，可以用二进制 bit01 串来代替，然后用与、或之类的计算，能够快速得到是否前置已完成，这样时间快很多，少了一个 for，虽然复杂度没变。其次，题目的拓扑关系比较简单，并没有考虑同分比较的情况。

感觉很现实啊，大作业如果占比少，不如复习让考试分高一点，会出现从月初就开始复习的情况。而当复习收益小，大作业分高的时候，会出现经常性的跳过。